

PLAN DE TEST

1. Objectifs

Ce plan de test définit la stratégie, le périmètre, les ressources et les critères d'exécution des tests pour l'Epic 1 → Access Management de l'application Tool Shop Demo (version with-bugs).

L'objectif principal est de valider l'ensemble des fonctionnalités liées à la gestion des accès utilisateur avant la mise en production, afin d'identifier les anomalies, incohérences et risques sur cette version.

Objectifs spécifiques :

- Valider que l'inscription (Register) fonctionne correctement avec les règles de validation définies
- Valider que la connexion (Login) est fonctionnelle et sécurisée
- Valider que la réinitialisation du mot de passe (Forgot your password) initie correctement le processus
- Valider que la déconnexion (Logout) détruit bien la session et redirige correctement
- Valider que le menu utilisateur connecté affiche les bonnes informations
- Identifier et documenter toutes les anomalies dans Jira

2. Périmètre des tests

2.1 Fonctionnalités incluses

Fonctionnalité	US liée	Type de test	Nombre de TC
Register (création de compte)	US-001	Automatisé	10 TC (dont 4 valeurs limites)
Login (connexion)	US-002	Automatisé	8 TC
Forgot your password	US-003	Auto (3) + Manuel (1)	4 TC
Logout (déconnexion)	US-004	Automatisé	4 TC
Menu utilisateur connecté	US-005	Automatisé	2 TC
Accessibilité (RGAA)	Transverse	Manuel	11 TC
TOTAL	5 US	27 Auto / 12 Manuel	39 TC

2.2 Fonctionnalités exclues

Élément exclu	Raison
Vérification email (Forgot password)	Accès boîte mail externe → hors portée Playwright → testé manuellement (TC-020)
Catalogue / Panier / Checkout	Couverts respectivement dans les Epics 2, 3 et 4
Tests de performance	Hors périmètre du projet portfolio
Tests de sécurité avancés (pentest)	Hors périmètre du projet portfolio

3. Stratégie de test

3.1 Types de tests

Type	TC concernés	Description
Smoke tests	TC-001, 002, 011, 017, 018, 023, 024	Vérification rapide que les fonctionnalités critiques fonctionnent — Register réussi, Login réussi, Logout. Exécutés en premier dans le pipeline CI.
Tests de régression	TC-003 à 010, 012 à 016, 019, 021 à 022, 025 à 028	Vérification des cas d'erreur, validations et comportements aux valeurs limites. Exécutés après les smoke tests dans le pipeline CI.
Tests de valeurs limites	TC-007, 008, 009, 010	Partition d'équivalence sur la validation du mot de passe → 8 caractères (invalide), 9 caractères (invalide), 10 caractères (valide), 11 caractères (valide).
Test manuel	TC-020	Vérification de la réinitialisation effective du mot de passe → nécessite accès à la boîte mail → hors portée Playwright.
Tests d'accessibilité	TCA11Y-001 à 011	Vérification RGAA (labels, contraste, navigation clavier, focus visible, messages de statut) avec WAVE, navigation clavier, inspection visuelle et le lecteur d'écran Orca.

3.2 Approche technique → Page Object Model

L'automatisation utilise le pattern Page Object Model (POM) pour séparer la logique de navigation de la logique de test, favoriser la réutilisabilité et faciliter la maintenance.

Fichier	Rôle
base.page.ts	Classe de base → navigate(), waitForPageLoad() → héritée par toutes les pages
register.page.ts	Locators du formulaire d'inscription + méthode fillForm()
login.page.ts	Locators du formulaire de connexion + méthode login()
navbar.page.ts	Locators du menu utilisateur + méthodes logout() et clickDropdown()
fixtures/test-data.ts	Données de test centralisées → TEST_USER (données utilisateur) + URLS (chemins de navigation)

3.3 Pipeline CI/CD

Les tests automatisés sont intégrés dans un pipeline GitHub Actions structuré en 3 jobs séquentiels :

Job 1 → Lint (ESLint) → vérifie la qualité du code TypeScript

Job 2 → Smoke tests (@smoke) → exécutés sur Chromium + Firefox si le lint passe

Job 3 → Regression tests (@regression) → exécutés si les smoke tests passent

Publication du rapport HTML → GitHub Pages après chaque exécution

4. Environnements de test

Paramètre	Valeur
Application testée	Tool Shop Demo (with-bugs) → version intentionnellement buguée pour l'entraînement QA
URL de test	https://with-bugs.practicesoftwaretesting.com/#/auth/login
Navigateurs	Chromium (Chrome) + Firefox → WebKit (Safari) exclu en raison d'une incompatibilité avec le site
Locale	EN → langue native du site
Stack d'automatisation	TypeScript + Playwright (@playwright/test v1.61.1)
Données de test	Compte de test dédié → credentials stockés dans les variables d'environnement CI (GitHub Secrets)
OS local	Ubuntu 24
OS CI	Ubuntu (GitHub Actions runner)
Node.js	v22.x
Repo GitHub	https://github.com/YassinePerso/QA__Portfolio_2__Typescript__Playwright

5. Critères d'entrée et de sortie

5.1 Critères d'entrée → conditions pour démarrer les tests

Critère	Vérification
L'environnement de test est accessible	with-bugs.practicesoftwaretesting.com répond correctement
Les cas de test sont rédigés et validés	39 TC documentés dans le fichier cas de test Sprint 1
Le compte de test est disponible	compte de test dédié créé et fonctionnel sur l'environnement with-bugs
Le projet Playwright est configuré	npm install effectué, playwright.config.ts validé
Les données de test sont centralisées	fixtures/test-data.ts → TEST_USER et URLS définis et validés

5.2 Critères de sortie → conditions pour clôturer les tests

Critère	Seuil / Condition
Tous les cas de test ont été exécutés	L'ensemble des 39 cas de test devront avoir été exécutés
Taux de succès minimum atteint	Le taux de succès devra atteindre $\geq 80\%$ de TC en statut Pass
Tous les bugs sont documentés	Chaque TC en Fail devra avoir un bug associé et documenté dans Jira
Aucun bug bloquant non documenté	Tout bug de sévérité ÉLEVÉE devra être ouvert dans Jira avant clôture
Le rapport d'exécution est produit	Le rapport d'exécution devra être produit, validé et archivé

6. Rôles et responsabilités

Rôle	Responsable	Responsabilités
Testeur QA	Yassine Boulakhrif	Rédaction des chartes, US, TC, plans de test → Exécution manuelle → Automatisation Playwright → Documentation des bugs dans Jira → Génération des rapports
Développeur	Équipe de développement	Correction des bugs documentés → Livraison des correctifs → Validation des corrections en environnement de test
Product Owner	Responsable produit	Validation des critères d'acceptance → Priorisation des bugs → Validation finale avant mise en production
CI/CD	GitHub Actions (automatisé)	Exécution automatique des tests à chaque push → Lint → Smoke → Regression → Publication du rapport HTML sur GitHub Pages

7. Risques anticipés

Les risques suivants ont été identifiés lors de la session d'exploration et doivent être pris en compte avant et pendant l'exécution des tests.

Risque identifié	Probabilité	Impact	Mitigation
Formulaire Forgot password potentiellement non fonctionnel	HAUTE	Utilisateur ne peut pas réinitialiser son mot de passe → bloqué hors de son compte	TC-020 Manuel + TC-021, 022 Auto pour valider le comportement
Incohérence validation mot de passe (message vs règle réelle)	MOYENNE	Utilisateur confus sur les règles réelles → peut créer un compte avec un mdp non conforme	TC-007 à TC-010 → partition d'équivalence aux valeurs limites
Menu utilisateur affiche des données incorrectes	MOYENNE	Dégradation de l'expérience utilisateur → perte de confiance	TC-027 → vérification du nom affiché après connexion
Vérification email (Forgot password)	HAUTE	Impossible d'automatiser la vérification de réception d'email	TC-020 exécuté manuellement

8. Planning

Phase	Activité	Durée estimée	Statut
Session exploratoire	Exploration + charte de test	90 minutes	Terminé
Rédaction documentation	US + Cas de test + Plan de test		Terminé
Exécution manuelle	39 TC exécutés manuellement		Terminé

Rapport d'exécution	Rapport + documentation Jira		Terminé
Automatisation Playwright	register.spec.ts + login.spec.ts + logout.spec.ts		Terminé
Intégration CI/CD	GitHub Actions pipeline		Terminé